

赛区评阅编号（由赛区组委会填写）：

2025 高教社杯全国大学生数学建模竞赛

承诺书

我们仔细阅读了《全国大学生数学建模竞赛章程》和《全国大学生数学建模竞赛参赛规则》(以下简称为“竞赛章程和参赛规则”,可从全国大学生数学建模竞赛网站下载)。

我们完全明白，在竞赛开始后参赛队员不能以任何方式（包括电话、电子邮件、网上QQ群、微信群等）与队外的任何人（包括指导教师）研究、讨论与赛题有关的问题。

我们知道，抄袭别人的成果是违反竞赛章程和参赛规则的，如果引用别人的成果或资料（包括网上资料），必须按照规定的参考文献的表述方式列出，并在正文引用处予以标注。在网上交流和下载他人的论文是严重违规违纪行为。

我们以中国大学生名誉和诚信郑重承诺，严格遵守竞赛章程和参赛规则，以保证竞赛的公正、公平性。如有违反竞赛章程和参赛规则的行为，我们将受到严肃处理。

我们授权全国大学生数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

我们参赛选择的题号（从 A/B/C/D 中选择一项填写）： A

我们的报名参赛队号（12 位数字全国统一编号）： 4321

参赛学校（完整的学校全称，不含院系名）： 同济大学

参赛队员(打印并签名): 1. 程浩然

2. 陆彦翔

3. 李博豪

指导教师或指导教师组负责人(打印并签名): 无

(指导教师签名意味着对参赛队的行为和论文的真实性负责)

日期: 2025 年 5 月 3 日

(请勿改动此页内容和格式。此承诺书打印签名后作为纸质论文的封面，注意电子版论文中不得出现此页。以上内容请仔细核对，如填写错误，论文可能被取消评奖资格。)

赛区评阅编号（由赛区组委会填写）：

2025 高教社杯全国大学生数学建模竞赛
编 号 专 用 页

赛区评阅记录（可供赛区评阅时使用）：

评 阅 人						
备 注						

送全国评阅统一编号（由赛区组委会填写）：

全国评阅随机编号（由全国组委会填写）：

（请勿改动此页内容和格式。此编号专用页仅供赛区和全国评阅使用，参赛队打印后装订到纸质论文的第二页上。注意电子版论文中不得出现此页。）

基于 A* 算法的仓库货架机器人路径规划与调度优化

摘要

现代仓库利用货架机器人提升拣选和配送效率，机器人在预设通道自主搬运货架实现自动化。多台机器人同时作业时易在有限通道空间发生路径冲突，影响仓库运行效率。需研究规划机器人行驶路径，合理安排任务执行时序，以避免碰撞拥堵，缩短任务总耗时。在这篇文章中，我们以 A* 算法为基础，实时调控多机器人的位置，在保证搬运效率的前提下，避免了可能出现的冲突问题。

关键字： 最短路径 A* 算法 时间层

目录

一、问题重述	2
1.1 问题的简化	3
二、模型的假设	3
三、符号说明	4
四、问题分析与模型设计	4
4.1 寻路算法	4
4.1.1 BFS（广度优先）算法	4
4.1.2 A* 算法	5
4.2 模型构建	7
4.2.1 设计思路	7
4.2.2 条件约束	8
4.2.3 A* 搜索主体流程	8
4.2.4 <i>expand_planner</i> 函数	10
4.2.5 流程说明	11
五、模型扩展	11
5.1 分层 A* 算法	11
附录 A 自定义类设计—python 源程序	12
附录 B 代码调用与输出结果—python 源代码	15

一、问题重述

多台货架机器人在仓库协同搬运任务时，需从不同位置取货并送至拣选工作站，期间易在通道产生路径冲突，引发碰撞或拥堵致仓库效率降低。所以要规划机器人避冲突行驶路径，合理安排任务执行时序，以避免碰撞且缩短任务总时长。这里我们以一个具体的操作场景开始，从小型仓库和少量机器人出发，探索在更大的时空规模下的多机器人如何实现最优的协同工作。

1.1 问题的简化

为了解决大规模仓储系统和多机器人协同工作的问题，我们从小规模问题开始探索，尝试找到统一的处理方法。

假设仓库如下图所示，由 6×6 的网格通道组成。初始时，机器人 1 位于货架 A 所在格子 (2,4)，机器人 2 位于货架 B 所在格子 (3,2)（行列坐标均以左上角为 (0,0) 原点）。两台机器人需要分别将各自的货架搬运至拣选站台 S（位于格子 (5,5)），完成配送任务。

机器人	起始位置（货架）	目标位置（拣选站台）
机器人 1（货架 A）	(2,4)	S (5,5)
机器人 2（货架 B）	(3,2)	S (5,5)

- （1）假定机器人每次移动相邻的一个格子（不可斜向移动）
 - （2）且任意时刻同一格子最多只允许一台机器人占据
 - （3）拣选站台 S 同一时间也只能接收一台机器人的货架交付
- 简化场景图：



图 1 简化场景图

二、模型的假设

- 忽略每个机器人的个体差异，即每个机器人完全相同；
- 假定所有可移动路径不存在非其他机器人的阻挡因素，如人为的拥堵；
- 机器人每次能且只能移动一格，且只能向上下左右移动，即不能走斜线；
- 认为每个机器人在任意时间都知道所有其他机器人的所在位置；
- 每个机器人每个时间占一个格子，即同一个格子中每单位时间最多只有一个机器人（机器人不能相撞）；
- 每个机器人取件与交付都不费时，即在 t 时间到了取件/交付点， $t+1$ 时刻就可以移动到附近的格子；
- 每个待取件都是匀质的，即它们都只占有一格，且每个机器人都可以取件；
- 每次交付时，即在 t 时刻有机器人到达交付点时，系统会发布下一个新的取件任务（即取件点的位置）；
- 下一个任务的位置是随机均匀分布的（除了交付台）。

三、符号说明

符号	意义
up	机器人向上运动
down	机器人向下运动
left	机器人向左运动
right	机器人向右运动
stay	机器人暂停运动
t	全局时间
f(n)	启发函数
g(n)	已付出的代价
h(n)	预估会付出的代价

四、 问题分析与模型设计

4.1 寻路算法

这里我们以最少时间为优化目标。

先考虑在仅有一个机器人，即不考虑碰撞的情况下，给机器人一个出发点，尝试得到该机器人到站台的最短路径。在图论中有几种常用的求最短路径的算法，如 **BFS**（广度优先）算法、**A*** 算法、**JPS**（跳点搜索）算法等

4.1.1 BFS（广度优先）算法

BFS 算法通过维护一个普通队列的方法，无偏好地加入新的可加入的节点，并对已加入队列的节点加上标记，直到找到终点，从而得到最短路径。我们可以发现，在很多情况下，这种算法需要加入大量无用的节点，从而影响了效率。如下图所示，对左上侧的探测是没有意义的。

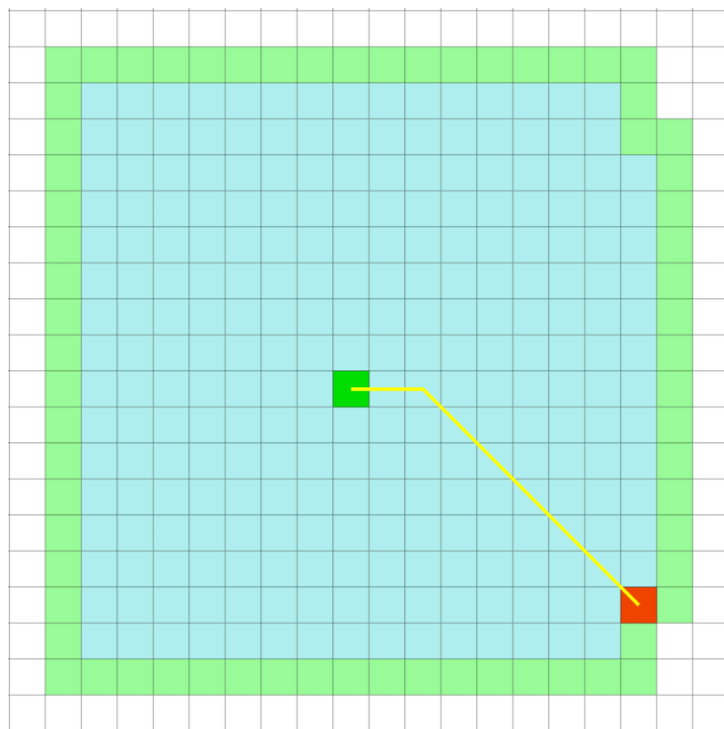


图 2 **BFS** 算法寻路 1

而且在有阻碍的情况下，BFS 算法的性能也会受到影响，如下图所示：

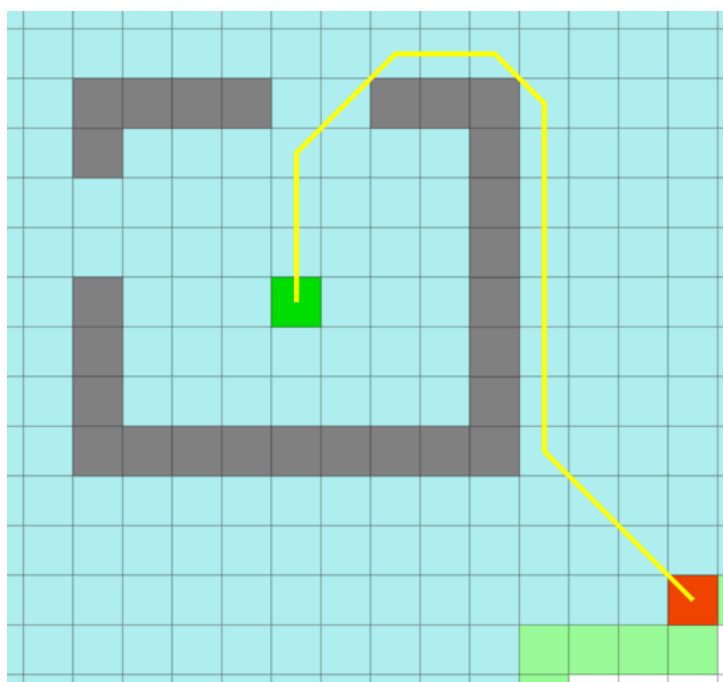


图 3 BFS 算法寻路 2

4.1.2 A* 算法

我们可以发现，BFS 算法效率低下的原因是对周围节点无偏好的加入，这导致有些对结果没有影响的节点也被加入进来了。如果我们在加入节点，对新节点的选择加以偏好（即加入启发式函数），每次只加入最有可能在最短路径上的点，这样就可以显著提高效率，如下图：

A* 算法的实现过程是通过建立和维护一个优先队列实现的。每一个待加入的节点都有自己的 $f(n)$ （启发式函数），启发式函数有两个部分组成： $g(n)$ 为已经付出的代价和 $h(n)$ 为预估会付出的代价。

$$f(n) = g(n) + h(n)$$

$g(n)$ 表示从出发点到该节点的最短距离， $h(n)$ 为预测的从该点到终点的最短距离。这里需要注意的是 $h(n)$ 的选取。这里取 $h^*(n)$ 为实际该点到终点的最短路径，已经有证明，只要满足如下条件，则必能通过 A* 算法找到最短路径。

$$h(n) \leq h^*(n)$$

这里值得注意的是当 $h(n)$ 越接近 $h^*(n)$ 时，A* 算法的效率越高。

在这种网格地图中有两种常用的 $h(n)$ 的取法，一种是取

$$h(n) = \sqrt{(x - x_0)^2 + (y - y_0)^2},$$

planner 旨在构建不同机器人在不同时间段下的分层情况，这样基于每一个不同的时间点都可以对机器人分布情况进行新的考虑，当最优化路径与当前机器人站位情况有重复时，将会考虑路径的重新规划以获得不产生冲突的最优路径表示。我们利用一个三维矩阵来表示 **planner**，分别指向规划过程中的 t, x, y 三个维度的数据内容，将每一单位时间的 x, y 表示作为一个 **layer** 进行数据的表示，在路径的寻找过程中，每一个 **layer** 依照 **agent** 的运动轨迹进行更新和传播，作为后续推导的依据。

在进行规划路径时，我们可以将每一个 **agent** 的单个移动步骤视为上、下、左、右和原地不动五种情况，通过遍历来查询最优路径，当遇到重复位置时需要重新进行规划，需要注意的是，已进行占位的机器人优先级高于新加入的机器人，重复此步骤直到完成所有时间层，就可以到达目标点。回溯每一个时间层的位置，即可得到最优路径。

4.2.2 条件约束

对每个机器人使用 **A*** 算法，然后判断是否为非法情况，在是非法情况时重新加入新节点。我们将所有的非法情况归类为下：

$$\left\{ \begin{array}{l} \text{发生非法碰撞} \\ \text{向上越界} \\ \text{向下越界} \\ \text{向左越界} \\ \text{向右越界} \end{array} \right.$$

为了避免这些非法情况，我们加入了一些约束条件。

$$\left\{ \begin{array}{l} \sum_{i=1}^N \sum_{d \in \{up, down, left, right, stay\}} \pi(pos(i, t, d) = (x, y)) x_{i,t,d} \leq 1 \\ y_{i,t,left} \leq \varepsilon(pos(i, t-1)_y > 1), \quad \forall i, t \\ y_{i,t,right} \leq \eta(pos(i, t-1)_y < n), \quad \forall i, t \\ x_{i,t,down} \leq \theta(pos(i, t-1)_x < m), \quad \forall i, t \\ x_{i,t,up} \leq \mu(pos(i, t-1)_x > 1), \quad \forall i, t \end{array} \right.$$

上面的 $\varepsilon(pos(i, t-1)_y > 1)$ 等为布尔函数，在内部表达式为真时结果为 1，反之则为 0。

在算法中，对于满足上述约束的节点正常使用 **A*** 算法处理，而在不满足约束时，则取其他的节点加入队列并置标记。

4.2.3 **A*** 搜索主体流程

1. 从优先队列中选取节点。在队列中所有节点按照总代价 ($g+h$) 进行排序，每一次选取总代价最小的节点作为当前需要扩展的节点。

```
current = heappop(heap)
```

2. 路径规划：如果当前节点的坐标 (`current.x`, `current.y`) 与目标坐标 (`targetx`, `targety`) 一致那么则说明已经找到了当前路径。在寻找到目标节点后，沿着父节点指针 `parent` 向上遍历，将所有经历的节点信息（时间 `t`，坐标 (`x`,`y`)，移动方向 `direction`）添加到 `path` 中，反转得到正向路径。

```
# 到达目标
if (current.x, current.y) == (targetx, targety):
    # 回溯并记录路径
    path = []
    while current:
        path.append((current.t, current.x, current.y, current.direction))
        current = current.parent
    path.reverse()
```

在确定路径后，调用 `expand_planner` 来确保 `planner_copy` 数组有足够的时间层来记录路径信息，同时更新 `planner_copy` 数组，标记路径节点被 `robotid` 占用，然后将路径中的移动方向添加到 `robot_actions` 列表，结束路径规划。

```
# 更新planner和robot_actions
if path:
    expand_planner(planner, time_needed: 0)
    planner[path[0][0]][path[0][1]-1][path[0][2]-1] = robotid

    for t, x, y, direction in path[1:]:
        expand_planner(planner, t, robotid)
        planner[t][x-1][y-1] = robotid
        if direction:
            robot_actions[robotid].append(direction)
```

进行完更新后，需要对后续时间层进行处理，将路径中最后一个节点的时间赋值给 `current_time` 用于作为处理后续时间层冲突的起始时间。检查在未来当前位置是否会引起冲突，如果已经被占用，就需要寻找相邻的空闲位置，使用和 3 中类似的方法，如果找到空闲位置就对其进行标记，并进行移动，跳出循环。如果无法找到空闲位置那么就将机器人的动作设置为停留，目标机器人继续被当前机器人占用（这一部分内容只是为了在形式上防止机器人的重叠，在实际工程问题中机器人的休息或者工作应当有下一个目标点进行指引，所以没有继续考虑）。

3. 子节点生成：由于当前节点 `current` 可能面临上、下、左、右四个方向的移动和停留操作，所以分别用 `[(0, 1), (0, -1), (1, 0), (-1, 0), (0, 0)]` 来表示 `x`, `y` 可能变化的五种情况，通过循环遍历这五种情况。检查边界条件，如果坐标节点不在仓库有效范围内 (`n*m`) 就跳过这种情况。然后检查这一种情况的下一个节点在 `next_t` 时刻是否被其他机器人占用，即是否满足 `planner_copy[next_t][next_x - 1][next_y - 1] == -1`，只有满足才是可用的。

```

# 处理后续时间层冲突
current_time = path[-1][0]
while current_time < len(planner)-1:
    current_time += 1
    expand_planner(planner, current_time)

# 检查未来时间层是否被占用
if planner[current_time][targetx-1][targety-1] != -1:
    # 寻找相邻空闲位置
    for dx, dy in [(0,1),(0,-1),(1,0),(-1,0),(0,0)]:
        nx = targetx + dx
        ny = targety + dy
        if 0 < nx <= n and 0 < ny <= m \
            and planner[current_time][nx-1][ny-1] == -1:
            planner[current_time][nx-1][ny-1] = robotid
            robot_actions[robotid].append(movements_rev[(dx, dy)])
            break
    else:
        # 无法找到空闲位置则保持原位
        planner[current_time][targetx-1][targety-1] = robotid
        robot_actions[robotid].append('stay')
else:
    planner[current_time][targetx-1][targety-1] = robotid
    robot_actions[robotid].append(movements_rev[(0, 0)])
return

```

```

# 生成子节点
for dx, dy in [(0,1), (0,-1), (1,0), (-1,0), (0,0)]:
    next_t = current.t + 1
    next_x = current.x + dx
    next_y = current.y + dy

# 边界检查
if 0 < next_x <= n and 0 < next_y <= m:
    expand_planner(next_t)

# 检查位置是否可用
if planner_copy[next_t][next_x-1][next_y-1] == -1:
    # 计算启发式值
    h = abs(targetx - next_x) + abs(targety - next_y)
    new_node = Node(next_t, next_x, next_y, current.g + 1, h,
                    current, movements_rev[(dx, dy)])

```

4. 更新 heap: 通过三维数组来检查未来位置是否可用，如果可用就进行更新。

```

# 直接检查三维数组标记
if planner_copy[next_t][next_x-1][next_y-1] == -1:
    planner_copy[next_t][next_x-1][next_y-1] = robotid
    heappush(heap, new_node)

```

4.2.4 *expand_planner* 函数

expand_planner 函数主要的作用是动态扩展用于记录机器人位置信息的三维数组 *planner_copy*，确保该数组有足够的时间层来记录机器人在整个规划路径过程中的位置变化。

1. 给该函数传入参数 *time_needed* 表示路径规划过程中需要的最大时间层，构建循环持续检查 *planner_copy* 的长度是否可以包含最大时间层，*the_planner* 是时间层的总

集合。

2. 当需要扩充时间层时, 创建一个新的二维数组 *new_layer*, 大小为 $n*m$, 初始元素被表示为-1。需要注意的是, 在每一层表示中, -1 代表该节点没有被机器占用。

```
# 动态扩展三维数组容量
def expand_planner(the_planner, time_needed, notchange=-1):
    while len(the_planner) <= time_needed:
        new_layer = [[-1 for _ in range(m)] for _ in range(n)]
        the_planner.append(new_layer)
```

4.2.5 流程说明

初始化: 开始后, 初始化仓库的行数 n 、列数 m 、交付站位置 *delivery_pos*、机器人初始位置 *robot_positions* 等参数, 接着初始化 *planner* 数组并在其中设置机器人的初始位置。

路径规划: 为每个机器人调用 *planrobot* 函数进行路径规划, 在规划过程中判断是否到达目标位置。若未到达, 生成子节点并检查其位置是否可用, 可用则计算启发值并添加到优先队列; 若到达, 则回溯路径, 更新 *planner* 和 *robot_actions*, 直至所有机器人路径规划完成。

可视化与交互: 创建 *GridVisualization* 对象开启可视化界面, 进入循环判断是否有用户操作。若有操作, 判断操作类型, 点击“Continue”按钮时更新机器人位置并绘制更新后的界面; 添加任务时添加任务并重新规划相关机器人路径; 若无用户操作则结束流程。

五、模型扩展

5.1 分层 A* 算法

在上述实验过程中，面对机器人稀疏，仓库空间狭小的情况，我们采用了 A* 算法来为所有机器人规划出了最优路径，然而在实际应用工程中，仓库的大小以及机器人的数量都要远远大于本题中的给定条件。尽管我们的实现方法可以完成对多机器人以及多路径问题的解决，但是这会导致相当巨大的计算开销和资源浪费。

当面对实际工程问题时，我们可以使用分层 A* 算法 (Hierarchical A*)，通过将路径规划问题分解成多个层次的子问题来提升路径搜索效率。我们可以将仓库划分为多个层次，比如将区域依照 5*5 进行划分，高层次被视为这些区域的中心。在高层路径搜索当中，我们不会考虑这些机器人在 5*5 方格内部的移动情况，而只会关心在这些 5*5 中心区域之间的转移模式。在确定最终抵达的高层次内容之后，我们在针对所有经历的区域进入底层进行路径的细化搜索，最后进行这些路径的链接，即可得到全局路径。

附录 A 自定义类设计-python 源程序

```
import tkinter as tk
from tkinter import simpledialog
'''
GridVisualization 使用方式如下:
1.初始化时使用GridVisualization(n, m, delivery_pos, robot_positions,
    robot_actions, tasks, action_pointers)
    n,m为仓库的行数和列数
    delivery_pos为交付站的位置
    robot_positions为机器人的初始位置
    robot_actions为机器人的动作序列,传入的是地址
    tasks是一个三维列表,第一个维度是机器人的编号,
    第二个维度是任务的队列,第三个维度存储了任务的信息,
    三个数据分别为任务的横纵坐标以及是否完成是否规划0表示未规划,1表示已经规划,2表示完成
    action_pointers存储了当前每个机器人的动作到了第几个任务
2.调用next可以让所有的机器人按着序列移动
3.其他函数均是私有函数

'''
class GridVisualization:
    UP = 'up'
    DOWN = 'down'
    LEFT = 'left'
    RIGHT = 'right'
    STAY = 'stay'
    def __init__(self, n, m, delivery_pos, robot_positions, robot_actions=None, tasks=None,
        action_pointers=None):
        self.n = n
        self.m = m
        self.delivery_pos = delivery_pos
        self.robots = robot_positions
        self.robot_actions = robot_actions if robot_actions is not None else [[] for _ in
            range(len(robot_positions))]
        self.tasks = tasks if tasks is not None else [[] for _ in
            range(len(robot_positions))]
        self.action_pointers = action_pointers if action_pointers is not None else [0] *
            len(robot_positions)
        self.task_added = False # 新增任务标记
        self.new_tasks_this_step = [] # 记录当前步骤新增任务的机器人ID
        self.root = tk.Tk()
        self.canvas = tk.Canvas(self.root, width=m*50, height=n*50)
        self.canvas.pack()
        self.draw_grid()
        self.draw_robots()
        self.draw_delivery()
```

```

# 初始化按钮
self.btn_continue = tk.Button(self.root, text='Continue', command=self.on_continue)
self.btn_continue.pack(side='left')
self.btn_add = tk.Button(self.root, text='Add Mission', command=self.on_add_mission)
self.btn_add.pack(side='right')

self.next()

def draw_grid(self):
    for i in range(self.n):
        for j in range(self.m):
            self.canvas.create_rectangle(j*50, i*50, (j+1)*50, (i+1)*50, outline='gray')

def draw_robots(self):
    # 清除所有旧的任务标记
    self.canvas.delete('mission')

    for idx, (x, y) in enumerate(self.robots):
        self.canvas.create_oval((y-1)*50+10, (x-1)*50+10, y*50-10, x*50-10, fill='blue',
                                tags=f'robot_{idx}')
        self.canvas.create_text((y-1)*50+25, (x-1)*50+25, text=str(idx), fill='white')
        if self.robot_actions[idx]:
            action = self.robot_actions[idx][0]
            action_text = {'up': '↑', 'down': '↓', 'left': '←',
                           'right': '→', 'stay': '•'}[action]
            self.canvas.create_text((y-1)*50+25, (x-1)*50+40,
                                    text=f"{idx}:{action_text}", fill='blue', font=('Arial', 12))

    # 只绘制第一个未完成的任务
    for task_idx, (task_x, task_y, status) in enumerate(self.tasks[idx]):
        if status == 0: # 未完成的任务
            self.canvas.create_rectangle((task_y-1)*50+5, (task_x-1)*50+5,
                                         task_y*50-5, task_x*50-5,
                                         outline='red', width=2, tags=f'mission')
            self.canvas.create_text((task_y-1)*50+25, (task_x-1)*50+25,
                                    text=f"M:R{idx}", fill='red')
            break

def draw_delivery(self):
    x, y = self.delivery_pos
    self.canvas.create_rectangle((y-1)*50+5, (x-1)*50+5, y*50-5, x*50-5, fill='green')

def update_robots(self, actions):
    for i in range(len(self.robots)):

```

```

        self.robot_actions[i].extend(actions[i] if isinstance(actions[i], list) else
                                      [actions[i]])

def on_continue(self):
    task_added = False
    for i in range(len(self.robots)):
        if self.robot_actions[i] and self.action_pointers[i] < len(self.robot_actions[i]):
            action = self.robot_actions[i][self.action_pointers[i]]
            x, y = self.robots[i]
            if action == self.UP and x > 1:
                self.robots[i] = (x-1, y)
                self.action_pointers[i] += 1
            elif action == self.DOWN and x < self.n:
                self.robots[i] = (x+1, y)
                self.action_pointers[i] += 1
            elif action == self.LEFT and y > 1:
                self.robots[i] = (x, y-1)
                self.action_pointers[i] += 1
            elif action == self.RIGHT and y < self.m:
                self.robots[i] = (x, y+1)
                self.action_pointers[i] += 1
            elif action == self.STAY:
                self.action_pointers[i] += 1
    self.canvas.delete('all')
    self.draw_grid()
    self.draw_robots()
    self.draw_delivery()
    # 重新绘制所有任务标记
    for robot_idx in range(len(self.tasks)):
        for idx, (x, y, status) in enumerate(self.tasks[robot_idx]):
            if status==0:
                self.canvas.create_rectangle((y-1)*50+5, (x-1)*50+5, y*50-5, x*50-5,
                                             outline='red', width=2, tags=f'mission_{idx}')
                self.canvas.create_text((y-1)*50+25, (x-1)*50+25, text=f'M:R{robot_idx}',
                                       fill='red')
                break
    self.root.quit()
    return task_added

def on_add_mission(self):
    task_added = False
    robot_idx = simpledialog.askinteger('添加任务', '输入机器人编号:', parent=self.root,
                                       minvalue=0, maxvalue=len(self.robots)-1)
    if robot_idx is not None:
        x = simpledialog.askinteger('添加任务', '输入目标x坐标:', parent=self.root,
                                    minvalue=1, maxvalue=self.n)
        y = simpledialog.askinteger('添加任务', '输入目标y坐标:', parent=self.root,

```

```

        minvalue=1, maxvalue=self.m)
    if x is not None and y is not None:
        if 0 <= robot_idx < len(self.tasks):
            self.tasks[robot_idx].append((x, y, 0))
            self.new_tasks_this_step.append(robot_idx)

        task_added = True
    self.task_added = task_added
    return task_added

def next(self):
    self.root.mainloop()
    current_tasks = self.new_tasks_this_step.copy()
    self.new_tasks_this_step.clear()
    self.task_added = False # 重置状态
    return current_tasks

def run(self):
    self.root.mainloop()

def complete_mission(self, robot_idx, mission_idx):
    if robot_idx < len(self.tasks) and mission_idx < len(self.tasks[robot_idx]):
        x, y, _ = self.tasks[robot_idx][mission_idx]
        self.tasks[robot_idx][mission_idx] = (x, y, 2) # 标记为完成
        self.canvas.delete(f'mission_{mission_idx}')
        self.canvas.create_rectangle((y-1)*50+5, (x-1)*50+5, y*50-5, x*50-5,
            outline='green', width=2, tags=f'mission_{mission_idx}')
        text_id = self.canvas.create_text((y-1)*50+25, (x-1)*50+25,
            text=f'R{robot_idx}', fill='green')
        self.root.after(1000, lambda: self.canvas.delete(text_id))
        return True
    return False

def __del__(self):
    if hasattr(self, 'root') and self.root:
        self.root.destroy()

```

附录 B 代码调用与输出结果—python 源代码

```
import copy

from grid_visualization import GridVisualization
from heapq import heappush, heappop

'''
在main.py中修改在初始化之后为每个机器人添加自己的路径，
采用A*算法让机器人从当前位置运动到delivery_pos,采用三维数组存储，
第二三个坐标对应着在棋盘上的坐标，第一个坐标表示时间，
下一刻的坐标为上一刻的坐标上下左右或者保持，
利用A*算法产生从初始地点到达delivery_pos的最短路径，
启发函数为曼哈顿距离。如果数组不够大那么添加一个新的数组。
每一次搜索后保留在三维数组中的结果是某刻在x,y的位置上有机器人占据。
不同的机器人不能占有相同的格子。A*搜索在添加了新的任务之后需要使用。
对于机器人x,如果添加了新的任务，需要先把货物运送到delivery_pos再到下一个目标地点。
因此一个机器人拥有两种运动方向，一种是去delivery_pos,一种是去任务点。
每当任务完成之后检查任务列表是否为空，如果任务列表为空的话，让机器人保持等待。
在A*搜索的过程中，如果发现前面的机器人是没有任务的状态并且机器人的任务列表为空，
那么就要为它在三维数组中找到一个位置放置它使它不挡路，如果发现它是有任务的，那么就绕开它。
'''

class Node:
    def __init__(self, t, x, y, g, h, parent=None, direction=None):
        self.t = t
        self.x = x
        self.y = y
        self.g = g
        self.h = h
        self.f = g + h
        self.parent = parent
        self.direction = direction

    def __lt__(self, other):
        return self.f < other.f

def planrobot(planner, robotid, targetx, targety, begintime, robot_actions):
    # 创建planner副本
    planner_copy = copy.deepcopy(planner)

    # 初始化移动方向和反向映射
    movements = {"up":(-1,0), "down":(1,0), "left":(0,-1), "right":(0,1), "stay":(0,0)}
    movements_rev = {(-1,0):"up", (1,0):"down", (0,-1):"left", (0,1):"right", (0,0):"stay"}

    # 查找机器人起始位置
```

```

start_t = begintime
start_x, start_y = -1, -1
for i in range(len(planner_copy[0])):
    for j in range(len(planner_copy[0][0])):
        if planner_copy[start_t][i][j] == robotid:
            start_x, start_y = i, j
            break
    if start_x != -1:
        break

# 初始化优先队列
heap = []
initial_h = abs(targetx - start_x) + abs(targety - start_y)
heappush(heap, Node(start_t, start_x+1, start_y+1, 0, initial_h))

# 动态扩展三维数组容量
def expand_planner(the_planner, time_needed, notchange=-1):
    while len(the_planner) <= time_needed:
        new_layer = [[-1 for _ in range(m)] for _ in range(n)]
        the_planner.append(new_layer)

# A*搜索主循环
while heap:
    current = heappop(heap)
    # 到达目标
    if (current.x, current.y) == (targetx, targety):
        # 回溯并记录路径
        path = []
        while current:
            path.append((current.t, current.x, current.y, current.direction))
            current = current.parent
        path.reverse()

        # 更新planner和robot_actions
        if path:
            expand_planner(planner, 0)
            planner[path[0][0]][path[0][1]-1][path[0][2]-1] = robotid

            for t, x, y, direction in path[1:]:
                expand_planner(planner, t, robotid)
                planner[t][x-1][y-1] = robotid
                if direction:
                    robot_actions[robotid].append(direction)

        # 处理后续时间层冲突
        current_time = path[-1][0]
        while current_time < len(planner)-1:

```

```

        current_time += 1
        expand_planner(planner, current_time)

        # 检查未来时间层是否被占用
        if planner[current_time][targetx-1][targety-1] != -1:
            # 寻找相邻空闲位置
            for dx, dy in [(0,1),(0,-1),(1,0),(-1,0),(0,0)]:
                nx = targetx + dx
                ny = targety + dy
                if 0 < nx <= n and 0 < ny <= m \
                    and planner[current_time][nx-1][ny-1] == -1:
                    planner[current_time][nx-1][ny-1] = robotid
                    robot_actions[robotid].append(movements_rev[(dx, dy)])
                    break
            else:
                # 无法找到空闲位置则保持原位
                planner[current_time][targetx-1][targety-1] = robotid
                robot_actions[robotid].append('stay')
        else:
            planner[current_time][targetx-1][targety-1] = robotid
            robot_actions[robotid].append(movements_rev[(0, 0)])

    return

# 生成子节点
for dx, dy in [(0,1), (0,-1), (1,0), (-1,0), (0,0)]:
    next_t = current.t + 1
    next_x = current.x + dx
    next_y = current.y + dy

    # 边界检查
    if 0 < next_x <= n and 0 < next_y <= m:
        expand_planner(planner_copy, next_t, robotid)

        # 检查位置是否可用
        if planner_copy[next_t][next_x-1][next_y-1] == -1:
            # 计算启发式值
            h = abs(targetx - next_x) + abs(targety - next_y)
            new_node = Node(next_t, next_x, next_y, current.g + 1, h,
                            current, movements_rev[(dx, dy)])

            # 直接检查三维数组标记
            if planner_copy[next_t][next_x-1][next_y-1] == -1:
                planner_copy[next_t][next_x-1][next_y-1] = robotid
                heappush(heap, new_node)

def get_final_positions(planner, time, robotid):

```

```

for i in range(n):
    for j in range(m):
        if planner[time][i][j] == robotid:
            return i+1,j+1
    return None,None
if __name__ == '__main__':
    n = 6 # 仓库的行数
    m = 6 # 仓库的列数
    delivery_pos = (6, 6) # 交付站的位置 (0-based索引)
    robot_positions = [(4,3), (3,5)]
    robot_actions = [[] for _ in range(len(robot_positions))]
    tasks = [[] for _ in range(len(robot_positions))]
    action_pointers = [0] * len(robot_positions)

    # 初始化planner三维数组
    planner = [[[-1 for _ in range(m)] for _ in range(n)]]
    for idx, (x, y) in enumerate(robot_positions):
        planner[0][x-1][y-1] = idx

    # 测试路径规划
    for i in range(len(robot_positions)):
        planrobot(planner, i, delivery_pos[0], delivery_pos[1], 0, robot_actions)
    action_pointers = [0] * len(robot_positions)
    grid=GridVisualization(n,m,delivery_pos,robot_positions,robot_actions,tasks,action_pointers)
    current_time = 0
    while True:
        # 检测机器人动作队列
        for i in range(len(robot_actions)):
            if not robot_actions[i]:
                # 获取当前位置
                x, y = -1, -1
                for row in range(n):
                    for col in range(m):
                        if planner[current_time][row][col] == i:
                            x, y = row+1, col+1
                            break
                # 检查下一时间步是否被占用
                if current_time+1 < len(planner):
                    if planner[current_time+1][x-1][y-1] != -1:
                        # 重新规划路径到相邻空闲位置
                        planrobot(planner, i, x, y, current_time, robot_actions)

        current_time += 1
        current_tasks = grid.next()
        # 处理新增任务
        print(current_tasks)
        print(tasks)

```

```

for robot_id in current_tasks:
    # 找到第一个未规划的任务并规划路径
    for task_idx in range(len(tasks[robot_id])):
        x, y, status = tasks[robot_id][task_idx]
        if status == 0:
            # 从当前位置开始规划到任务点再到交付点的路径
            start_pos = get_final_positions(planner, len(robot_actions[robot_id]),
                                             robot_id)

            # 规划到任务点的路径
            planrobot(planner, robot_id, x, y, len(robot_actions[robot_id]),
                      robot_actions)

            # 规划到交付点的路径
            planrobot(planner, robot_id, delivery_pos[0], delivery_pos[1],
                      len(robot_actions[robot_id]), robot_actions)
            print(robot_actions)

            # 更新任务状态为已规划
            tasks[robot_id][task_idx] = (x, y, 1)
            break
print(robot_actions)

```